

Act as a Principal Data Architect, JavaScript Engineer, SaaS Systems Architect, Security Engineer, and Product Designer.

Your task is to build Layer 1 of an AI Business Decision Intelligence Platform as a standalone HTML application.

The layer is called:

DATA ACQUISITION LAYER

Do not build the Business Operations Layer, Business Intelligence Layer, Digital Twin, simulation engine, or AI Decision Intelligence yet.

Purpose

The application must collect, validate, normalize, store, and expose real business data for future layers.

The processing flow must be:

Data source → ingestion → validation → normalization → duplicate detection → storage → event emission.

Existing Architecture Compatibility

The existing platform uses:

```
window.INFINICUS.bus.register(route, handler, schema)
```

```
window.INFINICUS.bus.call(route, payload)
```

```
window.INFINICUS.bus.emit(event, data)
```

```
window.INFINICUS.bus.on(event, listener)
```

Use the same route and event architecture so this layer can be connected to the existing platform later.

Do not alter or recreate the simulation engine.

Required Modules

Build:

1. Business identity management.
2. Universal business event envelope.
3. Data source registry.

4. Validation engine.
5. Data normalization engine.
6. Duplicate detection.
7. Storage adapter.
8. Manual sales entry.
9. Manual expense entry.
10. Customer entry.
11. Inventory movement entry.
12. CSV import preview.
13. CSV import execution.
14. Event log.
15. Validation error log.
16. Export and backup functionality.

Universal Event Structure

Every business event must follow this structure:

```
{ eventId, businessId, eventType, source, occurredAt, receivedAt, currency, payload, metadata }
```

Initial Event Types

Support:

business.created sale.created expense.recorded customer.created inventory.received inventory.consumed
inventory.adjusted employee.created supplier.created marketing.activity.recorded

Required EngineBus Routes

Register:

data.business.create data.business.get data.business.update

data.source.register data.source.list data.source.disconnect

data.event.ingest data.event.validate data.event.normalize data.event.list

data.sale.ingest data.expense.ingest data.customer.ingest data.inventory.ingest

data.import.csv.preview data.import.csv.execute data.import.status

Storage

Use:

- localStorage for settings and small profile records.
- IndexedDB for business events and large imports.
- A clean adapter interface for future Firestore or Supabase integration.

Do not hardcode external service credentials.

Security

Implement:

- HTML escaping
- Input sanitization
- Schema validation
- Numeric bounds
- Date validation
- Duplicate prevention
- Business ownership checks
- File size limits
- CSV row limits
- Safe error handling

Never inject raw imported CSV data into HTML.

User Interface

Use:

- Black background
- Neon green accents
- White typography
- IBM Plex Sans
- IBM Plex Mono
- Mobile-first responsive design
- Desktop left navigation
- Accessible forms
- Visible validation errors
- Clear success and failure states

Navigation:

Dashboard Businesses Manual Entry Imports Data Sources Event Log Errors Settings

Build Method

Build the project in small executable blocks.

For every block:

1. State its purpose.
2. State dependencies.
3. Provide the code.
4. Explain where to insert it.
5. Provide a test procedure.
6. Stop after completing the block.

Do not regenerate the entire HTML after every block.

Preserve all previous working code.

First Response

Do not write code immediately.

First provide:

1. Layer architecture.
2. Module dependency map.
3. Canonical data schemas.
4. Event namespace.
5. EngineBus route map.
6. Storage design.
7. Security controls.
8. Exact development block sequence.

After that, begin with Block 1 only.